BUDGETED CACHE REPAIR FOR CROSS-CONTEXT KV-CACHE REUSE

Haeyong Kang
Duksung Women's University

Chang D. Yoo
KAIST

arXiv:2610.02233v2 [cs.AR] 8 Oct 2026

ABSTRACT

Cross-context KV-cache reuse predicts a shared segment's keys and values under a new prefix instead of recomputing them, and has been reported to do so without quality loss. We find otherwise, and identify two problems. (1) **A hidden cost**: on MMLU and GSM8K, reuse costs substantial accuracy. (2) **A decision at the wrong unit**: no rule for deciding *whether* to reuse a cache removes that cost. What does help is choosing *which parts* of the cache to recompute, and the value of choosing well falls as the unit of choice grows: informed selection removes 49:5% of the cache error beyond chance at single rows (one token's keys and values), 10:6% at 64-token chunks, and nothing at the level of whole calls. **B**udgeted **C**ache **R**epair (**BCR**) acts at the unit where selection still pays. It drafts two tokens from the assembled cache, ranks cache rows by the attention those tokens pay them, and recomputes a fixed budget of rows exactly, in one of three layouts. The cost is paid rather than predicted away, and *the draft that fails as a gate succeeds as a selector*. BCR restores GSM8K to dense-prefill accuracy while still serving most calls from cache, and its best layout outperforms every reuse baseline's mean in the reference grid. The draft also beats a coin-flip selector at the same budget—a control prior evaluations lack.

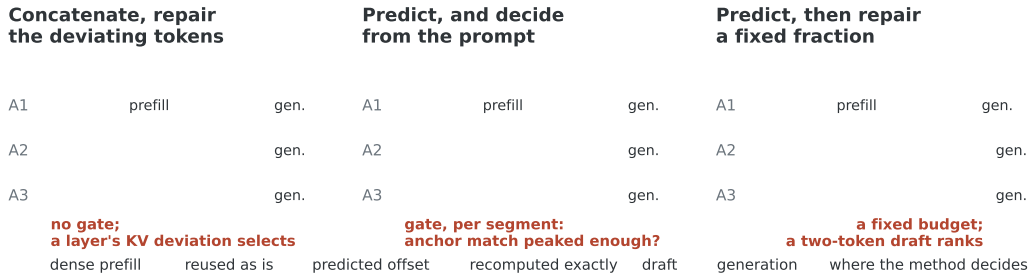
1 INTRODUCTION

A multi-agent LLM system processes the same context again and again. When three agents and an aggregator each receive the same question and each other's outputs, the question is prefilled four times and the first answer three times. This redundancy motivates cross-context KV-cache reuse (Kwon et al., 2023; Gim et al., 2024; Yao et al., 2025): rather than recomputing a segment's keys and values under every new prefix, a reuse method estimates whether the representations computed earlier are still valid. KVCOMM (Ye et al., 2026) does this with an online pool of *anchors*—segments stored together with the deviation a prefix change induced in them—and reports over 70% cache reuse and up to 7:8× faster time-to-first-token with no measurable quality loss.

A reused cache is not always safe to serve, so every reuse system needs a rule for deciding when its prediction can be trusted. CacheBlend (Yao et al., 2025), Figure 1(a), sidesteps the question by recomputing selected tokens instead of trusting the seams. KVCOMM instead applies a *gate*: once per reused segment, before generation begins, it thresholds the entropy of the segment's similarity to the stored anchors (§3). The gate's defining limitation, Figure 1(b), is that *it decides before any information about the answer exists*: it can inspect only the prompt and the predicted cache, so it uses the cache's geometry as a surrogate for whether reuse will change the answer.

We find that this prompt-side surrogate is poorly aligned with the errors reuse actually causes. Reuse costs substantial accuracy on two of three benchmarks, and the cost survives a check for anchor-pool warm-up. Tightening the gate does not remove it, and neither does choosing which calls to reuse: across five call-level allocation policies, including a random baseline, accuracy tracks the amount of reuse rather than the policy. Nor does deferring the decision until generation has begun, the fix that cache eviction uses (Li et al., 2024; Zhang et al., 2023; Wang et al., 2025): our generation-time gate OPENGATE, which decides from two draft tokens, loses 6:7 points to DENSE on 300 matched

Corresponding author: hykang@duksung.ac.kr.



(a) **CacheBlend.** No offset; a layer's deviation selects.

(b) **KVCOMM.** Offset from anchors; a gate refuses A3.

(c) **BCR (ours).** Offset kept; a draft selects a budget.

Figure 1: **Three ways to serve a shared segment.** Rows show three agents sharing one segment in execution order: A1 is the *first caller* and computes it exactly; A2 and A3 are *later callers* and can reuse it from cache. The horizontal axis denotes token position.

GSM8K problems (exact McNemar $p = 0.009$ on the problems where the two disagree; §4 gives the test form). Together these results say the problem is not *when* or *whether* to reuse but *what unit* the decision controls: a whole-call gate can only trust or reject an entire cache, even when the error sits in a small subset of its rows.

This motivates a different approach: *repair part of the cache instead of trusting all of it*. **B**udgeted **C**ache **R**epair (**BCR**) uses generation itself to find the parts of a reused cache most worth repairing. It drafts two tokens from the assembled cache, ranks cache rows by how much attention those tokens pay them, and recomputes exactly a fixed budget of the highest-ranked rows. We study three repair layouts, illustrated in Figure 1(c): fixed chunks, grown spans, and individual rows (§3). BCR pays the hidden cost directly, and pays it at a unit where informed selection can still reduce it.

Our contributions are as follows.

- **Budgeted Cache Repair.** We introduce BCR, a generation-guided method that uses a two-token draft to select, and then exactly recompute, a fixed budget of KV rows, in three repair layouts: fixed chunks, grown spans, and individual rows.
- **Accuracy recovery under reuse.** BCR recovers most of the accuracy reuse costs, restoring GSM8K to dense-prefill accuracy while keeping most of the reuse, and on each benchmark of the reference grid its best layout leads every reuse baseline's mean, KVCOMM and CacheBlend alike. Recovery is task-dependent: gains on MMLU are limited, and HumanEval has little degradation to recover.
- **What determines the error of reuse.** The value of informed selection falls as the unit of decision grows, from single rows to chunks to whole calls, and the cost itself is workload- or backbone-specific.

2 Related Work

Reuse across contexts. Prefix caching (Kwon et al., 2023) reuses a cache only under an exact prefix match. Prompt Cache (Gim et al., 2024) extends reuse to position-agnostic segments, and CacheBlend (Yao et al., 2025), Figure 1(a), repairs independently cached chunks by recomputing selected tokens. KVCOMM (Ye et al., 2026), Figure 1(b), takes a different route: it predicts the deviation a prefix change causes, using an online anchor pool, and decides per segment whether to reuse the cache. We study this formulation and ask whether such prediction preserves accuracy, including after the anchor pool has warmed up. Multi-agent pipelines (Du et al., 2023; Zhuge et al., 2024) make such reuse frequent; AgentPrune (Zhang et al., 2025) attacks the same cost from the communication side. Cache-compression methods face a related choice of what to keep, using positional rules (Xiao et al., 2024), reconstruction (Zweiger et al., 2026; Kim et al., 2026), or attention-based scores (Li et al., 2024; Zhang et al., 2023; Cai et al., 2025; Feng et al., 2026); recent work shows that such input-side criteria can be misaligned with task performance (Wang et al., 2025). BCR differs from all of these

in what it decides: it fixes a repair budget and chooses *what to repair*, rather than whether to trust the cache as a whole. Like CacheBlend it pays for selective recomputation, but it applies the idea to cross-context reuse and studies how the unit of repair changes its value; Figure 1(c) draws the contrast.

Selecting what to recompute. Recent work improves selective recomputation by ranking tokens with attention from the user query (Teng et al., 2026; Wang et al., 2026) or from a small model that approximates the target (Yang et al., 2026), and reports substantial gains at matched budgets. Draft passes that generate tokens before committing (Leviathan et al., 2023; Wang et al., 2025) likewise motivate generation-time signals for selection. These methods ask *which tokens to select* within a fixed formulation. We isolate two factors instead: what informed selection is worth relative to an uninformed baseline, and how that worth depends on the *unit* of selection. We therefore compare informed and random selection at the same repair budget, and vary the unit from whole calls to chunks to individual cache rows. That controlled view is the focus of BCR.

3 Budgeted Cache Repair

The reuse problem. A request runs on a directed acyclic graph of N agents. Agent i's prompt is assembled from a private system prompt S_i, the shared task T, and the outputs of its predecessors, so in the fully connected configuration studied here a segment such as T is prefilled $N+1$ times. Let $C(x \mid P)$ denote the KV cache of a token span x computed under prefix P, one *row* per token. Every entry of that cache depends on P through attention, so exact reuse requires an identical prefix—the condition prefix caching assumes (Kwon et al., 2023). KVCOMM (Ye et al., 2026) relaxes it by *predicting* the dependence. For a segment x already seen under some prefix P, it stores an *anchor*: the deviation

$$\Delta = C(x \mid P) - C(x \mid \varnothing); \tag{1}$$

between the segment's cache under P and its cache under no prefix at all. At serve time, a new segment's deviation is approximated by a similarity-weighted combination of stored Δ's, added elementwise to the cache.

The gate. An agent's prompt contains several such segments—the task, each predecessor's output—and each is decided separately: trust the approximation (*reuse*) or pay a full prefill for that segment (*dense*). KVCOMM decides *before generating any token*, from an input-side statistic. Let d_j be the L_2 distance in value space between the candidate segment and anchor j, collect the distances over the n available anchors as $d = (d_1, \ldots, d_n)$, and let $s = \mathrm{softmax}(-d)$. The rule thresholds the entropy of s:

$$\text{reuse} \quad \text{if} \quad H(s) \le \gamma \log_2 n, \qquad H(s) = -\sum_j s_j \log_2 s_j; \tag{2}$$

with a hard fallback to dense whenever $n \le 1$. A peaked s means the candidate closely matches one stored anchor, whose deviation should transfer. Since $\log_2 n$ is the largest entropy s can have, the threshold γ (default 0.3) is the fraction of maximum ambiguity the rule tolerates, and $\gamma \to 1$ recovers unconditional reuse. Equation (2) is the object this paper examines.

The repair. Predicting a cache instead of computing it costs accuracy, and no setting of the gate avoids that cost; §4.1 measures both. Reusing fewer caches, and choosing which, does not avoid it either: §4.2 rules that out and prices what choosing well is worth at each unit. What remains is to repair part of the reused cache rather than decide whether to trust all of it. As Figure 1(c) illustrates, BCR recomputes a fixed *budget* ρ of the cache's rows exactly and reuses the rest. Whenever KVCOMM serves a call from an assembled cache, BCR generates two tokens, rolls the draft back, and scores every cached row by the attention the draft paid it. Let $Q_{t,h}^{\ell}$ be the two query rows kept at layer ℓ, indexed by step t and head h, and K_h^{ℓ} the predicted keys. Row j scores $a_j = \sum_{\ell > 1} \mathrm{mean}_{t,h} \; \mathrm{softmax}(Q_{t,h}^{\ell} K_h^{\ell \top} / \sqrt{d_{\text{head}}})_j$, and a group of rows scores $\bar{a}_i = \mathrm{mean}_{j \in i} a_j$. The highest-scoring rows or groups, up to a budget of ρ of the cached rows, are recomputed exactly before generation begins. The keys scored are the predicted ones: the signal that decides what to fix is read from the tensors under suspicion. The draft is the same deferral OPENGATE applies to *whether* to reuse, applied instead to *which rows* to repair; App. B gives the procedure as Algorithm 2.

Table 1: **The budget: how large, in what layout, at what price.** (a) two agents, 150 matched GSM8K problems, 64-token chunks drawn by coin at four seeds so ρ is the only variable; *mean* and *spread* are the accuracy over those seeds and its range, max − min. KVCOMM is ρ=0, DENSE ρ=1. (b) rows recomputed per agent call at four agents. (c) time to the first token on a 4096-symbol synthetic segment at ρ=0.20, × the speedup against DENSE. Full tables in Tables 10, 11 and 15.

ρ	mean	spread
0.05	76.5	3.3
0.20	77.5	11.3
0.40	78.8	2.7
KVCOMM	77.3	–
DENSE	82.7	–

(a) The budget swept, four coin seeds

Method	MMLU	GSM8K	HEval
BCR-chunk	159	211	138
BCR-grow	174	221	152
BCR-single	167	216	154
CacheBlend, reuse	417	474	97
CacheBlend, cost	158	206	132

(b) Rows recomputed per call, four agents

Method	ms	×
DENSE	542.7	–
KVCOMM	72.9	7.63
BCR-chunk	190.1	2.85

(c) Time to first token, 4096 symbols

The layouts. Spending a budget at all recovers 7.9 of the 13.7 accuracy points reuse costs on GSM8K, and the rule that places the budget recovers 5.4 more (§4.3). How large the budget should be, Table 1(a)'s coin-drawn sweep cannot resolve: at ρ=0.20 its seeds span 11.3 accuracy points, wider than the 82.7 − 77.3 gap they bracket. What it leaves open is the layout the budget is spent in, and we test three, coarsest first:

- **BCR-chunk** divides the cache into fixed 64-token chunks and recomputes the highest-scoring ρ of them. Spans have a fixed length and sit on a grid.
- **BCR-grow** drops the fixed unit: it seeds on the best-scoring row and extends the span outward while a neighbour scores at least τ of the seed. Lengths follow the score surface, and positions are not aligned to anything.
- **BCR-single** drops grouping altogether and spends the budget one row at a time, the finest unit the budget admits.

One budget, three units. Unqualified, BCR means all three. Table 1(b) puts their row counts within 12% of the smallest (154 against 138 on HumanEval, the widest of the three; 9.4% on MMLU and 4.7% on GSM8K), so the three share one budget and differ in the unit they spend it in; comparing them isolates the unit, not the spend. Table 1(c) prices the budget in time: 190.1 ms to the first token, against reuse's 72.9 and DENSE's 542.7.

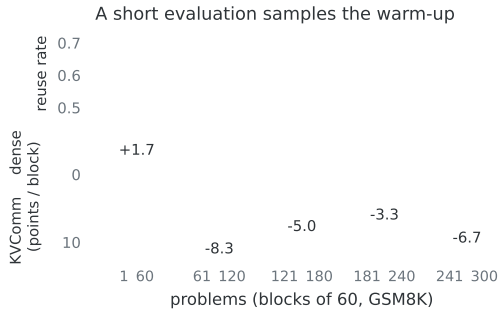
4 EXPERIMENTS

The measurement setting. GSM8K (Cobbe et al., 2021) and MMLU (Hendrycks et al., 2021) run on Llama-3.1-8B-Instruct (Grattafiori et al., 2024) and are scored by exact match and letter matching respectively; HumanEval (Chen et al., 2021) runs on Qwen2.5-Coder-7B-Instruct (Hui et al., 2024) and is scored by unit tests. Two configurations recur, and every result is labelled with the one it uses. The *reference* configuration follows the released implementation: roles cycle rather than stay pinned, and the shipped question budgets—150, 153 and 160 items—are run at 2 to 5 agents in Table 3. Its three-`MathSolver` GSM8K cell, with $N+1=4$ calls per request, is where §4.1 begins: 300 problems, the first 60 of which also serve to sweep γ. The *pinned* configuration assigns five agents fixed roles, over 300 matched GSM8K and MMLU problems in Table 5 or the 150 problems of Table 4(a); pinning concentrates divergence at the entry node, where §4.1 measures it. Statistical significance in this paper comes from these single runs over distinct items, never from Table 3. Decoding is greedy throughout, with 512 maximum new tokens, on one NVIDIA A100 80GB; anchor-pool hyperparameters stay at the repository defaults. App. G.1 gives the full environment and scoring rules.

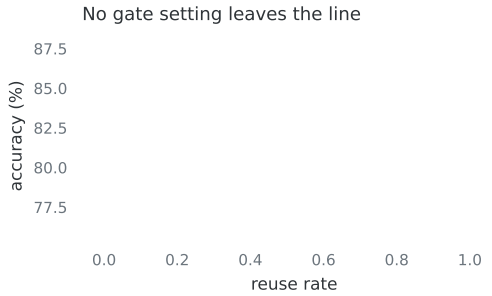
The statistical protocol. Within each comparison, every condition runs the *same* problems with the same decoding parameters, so every comparison is paired. We report the *discordant* split *b*=*c* before any *p*-value: *b* counts problems solved only by the first-named condition, *c* those solved only by the other, unless a table's caption fixes the orientation for its rows. An exact McNemar test on these paired disagreements then asks whether they favour one condition more often than chance would.

Table 2: **What the gate can and cannot do.** The first 300 GSM8K problems in the reference configuration. *b* counts problems DENSE solves and the row misses, *c* the reverse, *disc.* their sum; marks $p < 0.05$. *cost* is points against DENSE, taken from the discordant counts rather than the rounded accuracies. *rate* is the fraction of the 1200 agent calls served without a dense prefill.

		Accuracy vs. DENSE						Reuse
Method	reuse when	acc. (%)	95% CI	cost	*b*=*c*	disc.	*p*	rate
DENSE	never	85.7	[81.2, 89.2]	–	–	–	–	0.000
KVCOMM	$H(s) \le \gamma = 0.3 \log_2 n$	81.3	[76.5, 85.3]	−4.3	23/10	33	0.035	0.638
OPENGATE	conf. $\ge \theta = 0.5$	79.0	[74.0, 83.2]	−6.7	37/17	54	0.009	0.902
KVCOMM	$H(s) \le \gamma = 1.0 \log_2 n$	78.0	[73.0, 82.3]	−7.7	35/12	47	0.001	0.959



(a) **The anchor-pool warm-up.** The pool fills as the run goes on, so the first block reuses least and is the only one where reuse does not lose.



(b) **The gate sweep.** Every setting, and the answer-side gate, lands on the line DENSE and unconditional reuse define.

Figure 2: **What reuse costs, and what the gate can do about it.** GSM8K, $n = 300$ (Tables 2 and 6).

The split is more informative than marginal accuracy when two conditions differ on only a handful of problems. Results are reported per agent count rather than pooled across the grid: the four cells reuse the same problems, so pooling their pairs would treat repeated measurements as independent and inflate the effective sample size fourfold. Pooled splits therefore appear only descriptively, never as tests; App. H.1 records the ones we withdrew.

4.1 WHAT REUSE COSTS

We measure two things in order: whether predicting the cache instead of recomputing it costs accuracy, and whether that cost can be blamed on the *rule* that decides when to trust a prediction—the gate of Equation (2). Both measurements compare the same problems solved two ways, under the matched-pair protocol above.

The cost of reuse. Table 2 puts the cost at −4.3 points on 300 matched GSM8K problems in the reference configuration ($p = 0.035$ on 33 discordant pairs); pinned, the cost is 13.7 (§4.3). Unconditional reuse loses 7.7 ($p = 0.001$), and OPENGATE, the answer-side gate of App. A, loses 6.7 ($p = 0.009$). No setting of the gate, and no replacement we built for it, avoids the cost of predicting the cache; recomputing part of the prediction does (§3), while changing the decision about the whole cache does not. At $\gamma = 0.3$, reuse loses 23 problems that DENSE solves but *gains* 10 that DENSE misses, Table 2's 23=10: 33 problems change outcome for a net change of only 13, or 4.3 points. The approximated caches produce different answers, not occasional slips.

The warm-up effect. On the first 60 problems—a plausible pilot—reuse is *ahead* by 1.7 points, the opposite of the cost just measured. Figure 2(a) resolves this block by block, and the pattern follows the method's own dynamics rather than noise. The first block has the run's lowest reuse rate, 0.537, against 0.625 to 0.679 in later blocks, because a dense pass is what *materializes* an anchor: the realized deviation of Equation (1) needs the exact cache for its subtraction, so early requests build

Table 3: **What reuse costs across agent counts.** Reference configuration, $\gamma = 0.3$ (1.0 on HumanEval and on GSM8K's CacheBlend rows); 153, 150 and 160 items, matched per problem. **Bold** is the best score in a column excluding DENSE, shading the best BCR mean; *avg* carries no test. DENSE reuses nothing and the cost-matched pairing matches rows, so neither has a reuse row. Splits in Tables 13 and 14.

	MMLU					GSM8K					HumanEval				
# Agents	2	3	4	5	*avg*	2	3	4	5	*avg*	2	3	4	5	*avg*
Accuracy (%), HumanEval pass@1															
DENSE	74.5	75.2	75.2	75.2	75.0	82.7	86.0	84.7	84.7	84.5	87.5	86.2	85.6	85.0	86.1
KVCOMM	71.9	73.2	72.5	72.5	72.5	77.3	82.0	80.0	84.0	80.8	86.2	85.6	85.6	85.6	85.8
CacheBlend, reuse-matched	69.3	69.3	69.3	68.6	69.1	82.0	**86.7**	84.7	81.3	83.7	88.8	85.6	85.0	86.2	86.4
CacheBlend, cost-matched	72.5	71.2	69.3	73.9	71.7	**85.3**	83.3	83.3	82.0	83.5	85.0	84.4	85.6	85.6	85.2
BCR-chunk	74.5	**73.9**	72.5	75.2	74.0	82.0	86.0	85.3	**85.3**	**84.7**	86.2	86.2	85.6	85.6	85.9
BCR-grow	**77.1**	72.5	**75.8**	**75.8**	**75.3**	80.7	82.0	81.3	80.7	81.2	**89.4**	**88.1**	**88.1**	**88.1**	**88.4**
BCR-single	71.2	71.9	73.9	73.2	72.5	84.0	83.3	**86.0**	84.7	84.5	88.8	**88.1**	86.9	86.9	87.7
Reuse (%)															
KVCOMM	45.1	47.4	47.3	46.9	46.7	64.9	60.0	55.2	52.1	58.1	89.8	87.8	85.6	84.1	86.8
CacheBlend, reuse-matched	46.0	49.5	48.5	48.9	48.2	65.6	60.3	55.6	52.2	58.4	87.9	87.2	85.4	83.9	86.1
BCR-chunk	45.5	48.4	48.6	48.0	47.6	65.6	60.3	55.6	52.2	58.4	89.6	87.5	85.7	84.4	86.8
BCR-grow	46.4	50.8	52.3	52.0	50.4	63.8	58.8	53.9	50.6	56.8	89.2	87.8	86.2	84.9	87.0
BCR-single	45.5	48.2	48.6	48.1	47.6	64.9	59.8	55.1	51.9	57.9	89.8	88.6	87.4	86.2	88.0

the pool rather than draw on it. Every later block reuses more, and every later block is negative. A short evaluation therefore samples mostly the regime before the cost appears.

The gate sweep. If the loss came from misplaced trust in predictions, tightening the rule should recover quality. Table 2 sweeps it, and Table 9 sweeps it more finely on the first 60 problems, with latencies. The setting $\gamma = 1.0$ is not an extrapolation but the exact "never refuse" policy, since $H(s) \le \log_2 n$ makes the test unfireable there—which App. G.3 uses as a check that the setting took effect. Every setting stays below DENSE, and the cost deepens as reuse rises: -4.3, -6.7 and -7.7 accuracy points, at $p = 0.035$, 0.009 and 0.001. The gate moves the operating point along the one reuse–accuracy line that Figure 2(b) draws; it does not lift the line. The loss lives in the approximation itself, not in the rule that decides when to apply it.

The agent-count grid. Table 3 repeats the comparison at two to five agents, and DENSE leads KVCOMM in all eight MMLU and GSM8K cells. *No single cell is significant* at the repository's own sample sizes: Table 4(d) gives all twelve splits; the largest, GSM8K at two agents, is 17=9, $p = 0.169$. Pooling the counts is not a valid test (§4), so the grid gives direction, not magnitude; significance rests on the single $n = 300$ runs.

The same grid puts the repair at DENSE's ceiling, where the reuse baselines stop short. In Table 3's means—descriptive, since the four cells re-run the same problems—**BCR-chunk** takes GSM8K at 84.7, and **BCR-grow** takes MMLU at 75.3 and HumanEval at 88.4: level with DENSE's 84.5, 75.0 and 86.1, and clear of KVCOMM's 80.8, 72.5 and 85.8 and of CacheBlend's best, 83.7, 71.7 and 86.4. The three layouts spend one budget, and which of them leads depends on the benchmark (§4.3); some layout beats KVCOMM in all twelve cells and each CacheBlend pairing in eleven.

The comparison matches conditions on budget, and Table 3's GSM8K cell re-runs to the printed scores exactly. MMLU does not. Its analysis agent splices a live, uncached `en.wikipedia.org` fetch into the prompt, so every condition—DENSE included—depends on what Wikipedia said that day. Disabling retrieval makes the pipeline exactly reproducible, and that offline regime, documented in App. G.2, is what Table 3's MMLU column now reports. Measured offline, the cost is 2.0 to 2.7 points—three or four problems of 153 in Table 4(d)'s counts, none significant. The direction is unchanged, and the caveat applies to any MMLU number measured with roles cycling, the reference implementation's included.

The third benchmark. HumanEval behaves differently: reuse costs nothing. Its KVCOMM row runs at $\gamma = 1.0$, the setting whose reuse rate matches the published one, and across four cells it is level with DENSE at reuse above 84%, 28=26 in Table 4(d)'s pooled column. The tokenization term that §4.3 prices at 6.7 points on MMLU from Table 5(a) is measured there, not here, so we do not

Table 4: **The evidence behind the cost, the two nulls and the layout ordering.** (a) five agents, 150 matched GSM8K problems, roles pinned; *resid.* is the points earned beyond Equation (3). (b) six prompts; *gap* is the cache error informed selection removes beyond chance, as a share of the unrepaired 0.1054, *won* the prompts where it beat the coin. (c) the three layouts of §3, $b{=}c$ splits pooled over four agent counts. (d) is Table 13's cost rows, p in parentheses. Throughout b counts problems KVCOMM misses and the other solves. Full tables in Tables 7, 8 and 13.

Policy	reuse	resid.
Protect-the-Root	0.456	+0.9
Random	0.454	+1.6
Protect-the-Leaf	0.504	+0.4
DENSE	0.000	+0.0
KVCOMM	0.600	+0.0

(a) Allocation policies

unit (rows)	gap %	won of 6
1	49.5%	6
64	10.6%	5
1024	−5.7%	1

(b) Unit of choice

	chunk	grow	single
GSM8K	47/24	41/39	44/22
MMLU	21/12	54/37	50/50
HumanEval	4/3	29/12	27/15

(c) Layouts against KVCOMM

DENSE vs. KVCOMM	2 agents	3 agents	4 agents	5 agents	pooled
MMLU	16/12 (0.572)	15/12 (0.701)	18/14 (0.597)	18/14 (0.597)	67/52
GSM8K	17/9 (0.169)	11/5 (0.210)	15/8 (0.210)	9/8 (1.000)	52/30
HumanEval	8/6 (0.791)	7/6 (1.000)	5/5 (1.000)	8/9 (1.000)	28/26

(d) The grid's cells, $b{=}c$ (p), and their totals across the four counts

decompose this null. The cost is thus workload- or backbone-specific: present on two benchmarks, absent on the third. Where it is present, no rule avoids it—the premise from which §3's budget starts.

4.2 WHY A BUDGET, AND AT WHAT UNIT

The allocation policies. One might instead reuse fewer caches and choose which. Table 4(a) runs the policies of App. A on 150 GSM8K problems with roles pinned: Protect-the-Root and Protect-the-Leaf force a dense prefill at the entry node and at the deepest node, and Random gives up reuse at calls drawn by a biased coin; OPENGATE is the exception, which §4.1 measures in the reference configuration. The policies are distinguishable only by where they land on one line. Draw that line between DENSE at reuse 0 and unmodified KVCOMM at reuse 0.600, scoring 85.3 and 75.3 in Table 7; its slope is $(85.3 - 75.3)/0.600 = 16.7$:

$$\text{accuracy (\%)} = 85.3 - 16.7 \times \text{reuse rate}. \tag{3}$$

The three policies between the endpoints land on the line, none more than 1.6 points off (about half a standard error at $n = 150$; the endpoints define the line, so only these three residuals are free). The sharpest case is Random, which consults nothing—not the anchors, not the answer, not the topology—yet reaches Protect-the-Root's reuse to within 0.002 (0.454 against 0.456) and carries the larger residual of the pair. The two structural policies land at reuse 0.456 and 0.504, and a divergence measurement says which should win: conditioned on predecessors that already matched DENSE, the entry node diverges where the deepest does not, so the structural hypothesis predicts a gap. None resolves. The measurement was right about where error originates but wrong as a prescription: what determines the outcome is how much of the context was predicted, not where in the graph. One policy that fails to separate is weak evidence; four of different kinds—cache geometry, graph position twice, and nothing at all—make it a property of the problem. The cost is a function of how much you reuse, not of which calls. That claim is narrower than it sounds, for the reason the next measurement gives.

The unit of choice. Large reported gains for choosing which *tokens* to recompute sit awkwardly beside the null above for choosing which *calls* to reuse. The two are not in conflict: the unit of choice is the variable, and choosing well loses value as the unit grows. To show this, we start from a reused cache of a 2048-token segment, restore a nominal 20% of its rows, and vary only how those rows are grouped and chosen—by attention from the consuming context's decode queries, or at random. Error is the relative L_2 distance of the attention output from a dense cache's, pooled over layers and heads because a recomputed span is shared by all of them. Table 4(b) gives the curve. Informed selection removes 49.5% of the cache error beyond chance at single rows, 10.6%—a fifth of that—at 64-token chunks, and −5.7% at 1024, where it is nominally *worse*; the prompts it wins fall from 6 to 5 to 1 of

Table 5: **Where repair works, and what it is worth against a published selector.** Five agents pinned to one role, 300 matched problems. In (a) b counts problems KVCOMM solves at its shipped $\gamma = 0.3$ and the row misses, c the reverse, p in parentheses, marking $p < 0.05$; − marks a row with no split of its own—KVCOMM itself, the $^{\gamma}$ five-seed coin mean, and the token-matched control, which only MMLU needs. Shading marks the repair row. (b) pools the reference grid over four agent counts, descriptively (Table 14).

	GSM8K		MMLU	
Method	acc.	b=c (p)	acc.	b=c (p)
KVCOMM	71.0	–	51.7	–
BCR-chunk, coin	78.9$^{\gamma}$	–	50.0$^{\gamma}$	–
BCR-chunk, draft	84.3	9/49 (0.000)	54.7	12/21 (0.163)
token-matched DENSE	–	–	59.3	25/48 (0.010)
DENSE	84.7	10/51 (0.000)	66.0	21/64 (0.000)

(a) Cost and repair at $n = 300$

pairing	GSM8K	MMLU	HEval
reuse	25/31	39/69	29/26
cost	23/30	47/61	21/26

(b) CacheBlend vs. **BCR-chunk**

six. Table 8 carries the paired t behind those counts ($p = 0.08$ at 64) and notes that the budget drifts above the nominal 20% once chunks are coarse, so 256 and coarser are not budget-matched, pending a matched sweep. Two results that looked contradictory are thus one ordering, with our call-level null at one end and the literature's token-level gains at the other. The claim should be read with its unit attached: *allocation does not matter across agent calls*. The mechanism is concentration. Within a segment the attention mass is extremely uneven, so there is much to choose between; across calls, the candidates are whole prompts of the same shape.

The chunk unit is where KVCOMM's data structures can act, since its offset, anchor filter and re-rotation all assume a contiguous span, as App. A.2 details. The curve therefore predicts that at 64-token chunks a selector keeps about a fifth of its single-row advantage, and that BCR should gain from repairing at all more than from repairing the right places. The task level bears out the ordering but not the fraction: in Table 5(a) the budget alone is worth 7.9 points (the coin's 78.9 over KVCOMM's 71.0) and the selector a further 5.4 (84.3 over that coin), so the curve bounds the selector's value only loosely. Whether the curve's ordering by unit survives at the task level is the question of Table 4(c), which §4.3 reads.

4.3 WHAT THE REPAIR RECOVERS

DENSE prefills every segment and is the ceiling in the sense that matters: no budget $\rho < 1$ can improve on $\rho = 1$. KVCOMM is the method under study. CacheBlend (Yao et al., 2025), reproduced from its released rule inside this harness so that only its selection rule differs from BCR's, is the published selective-recomputation competitor, and a coin at the same budget is the control. The gate is $\gamma = 0.3$ on the Llama benchmarks and $\gamma = 1.0$ on HumanEval; every repair takes $\rho = 0.20$, with $\tau = 0.25$ for **BCR-grow**. Within each comparison the selection condition is therefore the only variable, with one exception: GSM8K's CacheBlend pairings run at $\gamma = 1.0$ against the other conditions' 0.3, a mismatch that App. D shows runs against CacheBlend.

The GSM8K recovery. In the pinned configuration, Table 5(a) has the repair recovering $84.3 - 71.0 = 13.3$ of the $84.7 - 71.0 = 13.7$ points reuse costs, level with DENSE. Against KVCOMM its discordant split is 9=49 in that table's orientation (9 only KVCOMM solves, 49 only the repair; $p < 10^{-3}$). The run's reuse rate is 0.629, at which Equation (3), drawn through Table 7's $n = 150$ endpoints, predicts $85.3 - 16.7 \times 0.629 = 74.8$; the repair sits 9.5 points above the line that allocation could only slide along, because its budget is spent below the call rather than among calls. The direction repeats in Table 3's reference grid: **BCR-chunk** closes the gap in every GSM8K cell and is level with DENSE at 26=25 pooled, as App. E shows. App. H.1 withdraws a pre-registered pooled test; the $n = 300$ run over distinct items carries this claim.

The random-selection control. At the same budget we run a coin—the control the selective-recomputation literature competes without. One run scored 82.3%, and we first read its two-point gap to the draft-scored repair as a null. Five seeds withdraw that reading. Across seeds accuracy spans

74:3 to 82:7% over the five runs App. H lists. The draft-scored repair is above all of them, three of the five paired tests separate the two, and Table 5 reports the mean. On GSM8K at 64-token chunks the selector is worth 84:3 − 78:9 = 5:4 points, more than §4.2's curve predicted.

The layout ordering. Every repair layout is tested against KVCOMM in every cell of Table 13's grid. Of the 36 tests exactly one is significant—**BCR-single** on GSM8K at two agents, 14=4, $p = 0{:}031$—and it does not survive App. H.1's correction for the comparisons it sits among. What the grid does give is direction, in the pooled splits of Table 4(c)—descriptive, because the four cells re-run the same problems. On GSM8K single rows lead KVCOMM and grown spans do not move; on MMLU the ordering inverts and weakens. HumanEval has no cost to recover: summing Table 4(d)'s 8=6, 7=6, 5=5 and 8=9 puts DENSE at 28=26 against KVCOMM, and no repair damages it.

The published selector. Table 3 pairs CacheBlend at matched cost and at matched reuse, with Table 1(b) counting the rows. Neither pairing separates from **BCR-chunk** in any cell of Table 14 (p from 0:110 to 1:000), and Table 5(b) pools them. Ranking individual *tokens* by a layer-1 probe buys nothing here over 64-token chunks ranked by a two-token draft, though §4.2's curve predicts a gap. Table 3 also has BCR above DENSE in sixteen of its thirty-six cells, and none of them shows repair beating exact computation: the splits are level, and tokenization makes a repaired call a *different* input, free to land on either side of DENSE, as App. E explains.

The MMLU tokenization term. MMLU is graded by letter matching, and four stricter extractors put its 43-problem cost at 28–31, so its accuracies are upper bounds. In the pinned configuration, Table 5(a) shows MMLU recovering nothing significant: 3:0 points (54:7 against 51:7) on 33 discordant pairs (12+21), $p = 0{:}16$, while both reuse ($p < 10^{-3}$) and repair sit far below DENSE. The repair ran as designed—1,253 calls repairing 95,425 rows, five skipped—and did not help. The reuse path concatenates per-segment IDs, and that sequence need not equal the chat template's tokenization of the same text. A token-matched control keeps the sequence but removes the approximation, by forcing every call to a dense prefill of the reuse path's merged IDs. It scores 59:3%, which splits the gap in two: 6:7 points from tokenization (66:0 − 59:3) and 7:6 from approximation (59:3 − 51:7; 25=48, $p = 0{:}010$). **BCR-chunk** recovered 3:0 of the 7:6 and cannot touch the 6:7, because those points are not in the cache but in which tokens the cache is *of*. On GSM8K the entry tokenizations coincide, the term vanishes, and repair recovers nearly the whole gap—the benchmark asymmetry in one line. Any reuse system that concatenates per-segment IDs pays this tax.

What the repair buys. At a budget of a fifth of the cache, BCR returns GSM8K to DENSE while 63% of calls are still served from cache, and by Table 1(c) reaches the first token 2:85× sooner than DENSE does. In Table 3 a BCR layout holds the best mean short of DENSE in thirteen of fifteen columns and in every *avg* column, and the best layout's average is at or above DENSE's on all three benchmarks. CacheBlend leads no average at either pairing, and its reuse-matched pairing recomputes more than twice **BCR-chunk**'s rows per call. The draft is what pays for this: at the same budget it beats a coin by 5:4 points, and no layout damages the benchmark that has nothing to recover. Where it recovers little, on MMLU, almost half of the gap is one no budget could reach, because it is not in the cache.

5 DISCUSSION AND CONCLUSION

Cross-context KV-cache reuse is reported to be lossless. On two of three benchmarks we find otherwise. With roles pinned, reuse costs 13:7 points on GSM8K and 14:3 on MMLU (9:3–10:3 after re-grading), while HumanEval shows no measurable loss. Nor can it be avoided by choosing which calls to reuse: the value of informed selection falls from 49:5% to 10:6% to 0 as the unit of decision grows. It can be repaired: a fixed budget of exact recomputation lets BCR restore GSM8K to DENSE accuracy while keeping most of the reuse (reuse rate 0:629), and two of the three layouts reach that level; on every benchmark of the reference grid the best layout leads every reuse baseline's mean. On MMLU, however, only part of the gap is in the cache at all.

The key finding is that *where* the decision is made matters as much as *whether* it is. Our evidence spans three benchmarks and two backbones, but a single topology and a single chunk size. BCR also trades latency for accuracy: its 2:85× speedup in Table 1(c) is below reuse's 7:63×, a trade App. J

weighs. Testing the grid's ordering cell by cell, carrying the protocol to other reuse systems, and closing that latency gap are the work the same appendix lays out.

AI USE STATEMENT

We used generative AI tools solely for language editing, including grammar correction and concise rewriting. All research, ideas, and manuscript content were developed by the authors. We reviewed all AI-assisted edits and take full responsibility for the final content.

REFERENCES

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=ayi7qezU87.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *Advances in Neural Information Processing Systems*, 38:113152–113188, 2026.

In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338, 2024.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report, 2024. URL https://arxiv.org/abs/2409.12186.

Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W Lee, Sangdoo Yun, and Hyun Oh Song. Kvzip: Query-agnostic kv cache compression with context reconstruction. *Advances in Neural Information Processing Systems*, 38:167563–167591, 2026.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International conference on machine learning*, pp. 19274–19286. PMLR, 2023.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.

Xin Teng, Canyu Zhang, Shaoyi Zheng, Danyang Zhuo, Tianyi Zhou, and Shenji Wan. Infoflow kv: Information-flow-aware kv recomputation for long context, 2026. URL https://arxiv.org/abs/2603.05353.

Shihao Wang, Jiahao Chen, Yanqi Pan, Hao Huang, Yichen Hao, Xiangyu Zou, Wen Xia, Wentao Zhang, Chongyang Qiu, and Pengfei Wang. Prophetkv: User-query-driven selective recomputation for efficient kv cache reuse in retrieval-augmented generation, 2026. URL https://arxiv.org/abs/2602.02579.

Yixuan Wang, Shiyu Ji, Yijun Liu, Yuzhuang Xu, Yang Xu, Qingfu Zhu, and Wanxiang Che. Lookahead q-cache: Achieving more consistent kv cache eviction via pseudo query. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 34146–34162, 2025.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pp. 21875–21895, 2024.

Bin Yang, Qiuyu Leng, Jun Zeng, and Zhenhua Wu. Cacheclip: Accelerating rag with effective kv cache reuse, 2026. URL https://arxiv.org/abs/2510.10129.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*, pp. 94–109, 2025.

Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, et al. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *Advances in Neural Information Processing Systems*, 38:17882–17928, 2026.

Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. In *International Conference on Learning Representations*, volume 2025, pp. 75389–75428, 2025.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in neural information processing systems*, 36:34661–34710, 2023.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Language agents as optimizable graphs. *arXiv preprint arXiv:2402.16823*, 2024.

Adam Zweiger, Xinghong Fu, Han Guo, and Yoon Kim. Fast kv compaction via attention matching, 2026. URL https://arxiv.org/abs/2602.16284.

A Allocation Policies in Detail

The policies we test were not chosen to be exhaustive but to be *different in kind*, so that a common outcome cannot be blamed on a common blind spot. They are organised by the unit they allocate over.

Table 6: **Accuracy and reuse rate by block.** DENSE against the shipped default ($\gamma = 0.3$) in blocks of 60 GSM8K problems, with that block's reuse rate. The shaded first block—what a 60-problem evaluation would see—is the only one where reuse does not lose, and the one with the lowest reuse rate.

Problems	reuse rate	DENSE	KVCOMM $\gamma = 0.3$	Δ
1–60	0.537	80.0	81.7	+1.7
61–120	0.625	88.3	80.0	−8.3
121–180	0.679	90.0	85.0	−5.0
181–240	0.675	83.3	80.0	−3.3
241–300	0.675	86.7	80.0	−6.7
1–300	0.638	85.7	81.3	−4.3

Table 7: **Allocation policies on one line.** Five agents, 150 matched GSM8K problems, roles pinned; rows sorted by reuse rate. *pred.* is what Equation (3) predicts from the reuse rate alone, *resid.* the points earned beyond it. p is an exact McNemar test against DENSE; marks $p < 0.05$. Both columns use the unrounded scores, so redoing them from the printed ones can shift the last digit.

Policy	decides from	reuse	acc. (%)	95% CI	p vs DENSE	pred.	resid.
DENSE	no reuse	0.000	85.3	[79, 90]	–	85.3	+0.0
Protect-the-Root	structural	0.456	78.7	[71, 84]	0.013	77.7	+0.9
Random	information-free	0.454	79.3	[72, 85]	0.064	77.8	+1.6
Protect-the-Leaf	structural	0.504	77.3	[70, 83]	0.023	76.9	+0.4
KVCOMM $\gamma = 0.3$	entropy gate	0.600	75.3	[68, 82]	0.006	75.3	+0.0

A.1 ALLOCATING WHOLE AGENT CALLS

P1: the entropy gate (what KVCOMM ships). The baseline policy is Equation (2), swept from the shipped $\gamma = 0.3$ to 1.0.

P2: OPENGATE, a gate on the answer's opening. Eviction's fix for a miscalibrated prompt-side score is to defer the decision past the first generated tokens. OPENGATE transplants that fix: serve optimistically, generate k tokens on the assembled cache, decide from the confidence of those k distributions, and pay a dense prefill when the draft is not trusted.

The literal transplant does not carry over, and it is worth saying why. In eviction the exact cache exists for free at the end of prefill, so deferring costs only k decode steps; in reuse the exact cache is *never computed*, and obtaining it costs the prefill that reuse exists to avoid. What can be moved is the *decision*, not the approximation.

P3 and P4: structural allocation. A third kind decides from the shape of the request rather than from a signal, before it starts. *Protect-the-Root* forces a dense prefill at the entry node; *Protect-the-Leaf* does the same at the deepest node. They force the identical number of dense calls and differ only in *which* node is protected, which makes them a matched pair.

P5: random allocation. Give up a fixed fraction of reuse opportunities by biased coin flip, with the probability chosen so the achieved reuse rate lands on top of the structural policies.

A.2 ALLOCATING BELOW THE CALL: BCR-CHUNK

Chunks rather than tokens. The replacement policies above are all-or-nothing per agent call, at or above the unit of the per-segment gate they replace, and §4.2's curve points at a finer one. Per-row selection is where the choice is worth most, and it is what the selective-recomputation literature for RAG does. It is also what KVCOMM's data structures cannot express. The offset of Equation (1) is a position-indexed elementwise tensor, anchors are filtered by a length comparison, and re-rotation applies one scalar positional shift to a segment; all three assume the reused unit is a dense, contiguous,

Table 8: **Selection value by unit of choice.** One 2048-token segment, six prompts, varying only how rows are grouped and picked. Error is the relative L_2 distance of the attention output from a dense cache (0.1054 unrestored); *gap* is what informed selection removes beyond chance; the last column gives prompts where informed won and a paired t over the six. The budget keeps $\max(1, \text{round}(0.2 n_{\text{chunks}}))$ chunks, so the last three rows are not budget-matched.

unit of choice	budget	attention	random	gap	gap / no-repair	prompts (p)
1 (single row)	20.0%	0.0375	0.0896	+0.0521	49.5%	6=6 (0.01)
16	20.3%	0.0650	0.0851	+0.0201	19.0%	6=6 (0.02)
64	18.8%	0.0763	0.0875	+0.0112	10.6%	5=6 (0.08)
256	25.0%	0.0780	0.0880	+0.0100	9.5%	4=6 (0.28)
1024	50.0%	0.0691	0.0631	−0.0060	−5.7%	1=6 (0.69)
2048 (whole segment)	100.0%	*both restore everything*		0	0%	–

length-typed span. A contiguous chunk satisfies every one of those assumptions, and recomputing one is an ordinary prefill rather than a custom attention path.

The policy. On every call KVCOMM serves from an assembled cache, BCR inserts three steps before generation. A prompt-side score would be free at that moment—KVCOMM's own gate is one such score—but any such score ranks cached rows by the geometry of the cache rather than by what the answer turns out to need. We therefore spend k decode steps on a draft instead.

1. **Draft.** Generate $k=2$ tokens on the assembled cache and keep the last k query rows. The draft is then discarded: every layer is truncated back to its pre-draft length and the token counter restored. Repair and real generation therefore both begin from the cache the call was served with. The k drafted tokens are decoded a second time, now against the repaired cache, and need not come back the same. OPENGATE handles its draft the other way: it keeps the draft and continues from the returned cache (App. I). That is the case in which "no extra forward pass" holds; for BCR it does not. The served cache stops one token short of the prompt ($T = |x| - 1$), so at $k=2$ the kept rows are the queries of the prompt's final token and of the first generated one: one prompt-side, one answer-side. We fix $k=2$ and never sweep it, so §4.3's estimate bounds what one answer-side query row is worth, not what a longer draft would be.
2. **Score.** The rule of §3, with grouped-query keys expanded to query heads. Scoring a chunk by the mean of its rows' scores keeps a short trailing chunk from being penalised for its length, and unlike an eviction mask the selection cannot differ per head.
3. **Repair.** Recompute the highest-scoring ρ of the chunks exactly against the cache, and resume.

The skip rule, the recompute path, the budget's denominator and the correctness check are stated once in App. B, with Algorithm 1.

The random control. Repairing chunks and *choosing* them are separable, and §4.2 predicts the choice is worth much less at the chunk unit than at the row unit. We therefore run the same policy with the chunks picked by coin at an identical budget. Whatever separates the two rows is the draft's contribution; whatever they share belongs to repair itself. The coin condition also skips the draft entirely, so it spends k fewer decode steps per repaired call: a null between the rows would have been a dominance rather than a tie. A single coin run had suggested such a null; the five-seed control of §4.3 replaces it, so the draft is not free to remove, and App. C prices what the timed rows omit.

B SELECTIVE RECOMPUTATION

The recompute path. Repairing a chunk exactly requires everything before it, so recomputing it in isolation is wrong and recomputing the prefix again saves nothing. We run only the selected positions forward while they attend to the cache already present. Each layer does four things. The selected rows' hidden states go through the block's projections and rotary embedding. Their new keys and values overwrite their own slots *before* attention is taken, so a repaired row attending to another repaired row sees the corrected value. Their queries then attend over the whole cache under a causal mask built from absolute positions. The block output feeds the next layer. Layers proceed in